

MANUAL DEL PROCESO
ELABORACIÓN DE ESPECIFICACIONES TÉCNICAS

ÍNDICE DE CONTENIDOS

1	OBJETIVO	3
2	ALCANCE Y ÁMBITO DE APLICACIÓN	3
3	NORMAS GENERALES DE OPERACIÓN	3
3.1	BASE LEGAL	3
3.2	POLITICAS DEL PROCESO.....¡Error! Marcador no definido.	
4	DESCRIPCIÓN DEL PROCESO.....	5
4.1	DIAGRAMA DE RELACIONAMIENTO DE LOS PROCESOS	5
4.2	DIAGRAMA DE FLUJO.....	6
4.3	DESCRIPCIÓN DE ACTIVIDADES	7
5	INDICADORES DE PROCESO.-	7
6	GLOSARIO DE TÉRMINOS Y ABREVIATURAS	8
7	ANEXOS	8

IDENTIFICACIÓN DEL DOCUMENTO

MACROPROCESO:	INFRAESTRUCTURA/GESTIÓN DE TECNOLOGIA
PROCESO:	ELABORACIÓN DE ESPECIFICACIONES TÉCNICAS DE ARQUITECTURA DE LA SOLUCIÓN
SUBPROCESO:	DISEÑO Y DESARROLLO DE SISTEMAS ACADÉMICOS
RESPONSABLE:	LIDER TECNICO

1 OBJETIVO

Definir y documentar las especificaciones técnicas, arquitectura de software y reglas de negocio para el desarrollo e implementación del sistema de gestión académica integral denominado "PROYECTO NOTAS", garantizando una solución robusta, escalable y alineada a la lógica del sistema educativo ecuatoriano.

2 ALCANCE Y ÁMBITO DE APLICACIÓN

Comprende desde el análisis de la necesidad planteada por la unidad académica (gestión de años lectivos, matrículas, parciales, actividades, evidencias y promedios), hasta la emisión de las especificaciones de la solución tecnológica utilizando el ecosistema basado en TypeScript (Nest.js, Next.js, Prisma, PostgreSQL).

El ámbito de aplicación del proceso es a nivel institucional para la comunidad educativa (Administradores, Profesores y Estudiantes).

3 NORMAS GENERALES DE OPERACIÓN

3.1 BASE LEGAL

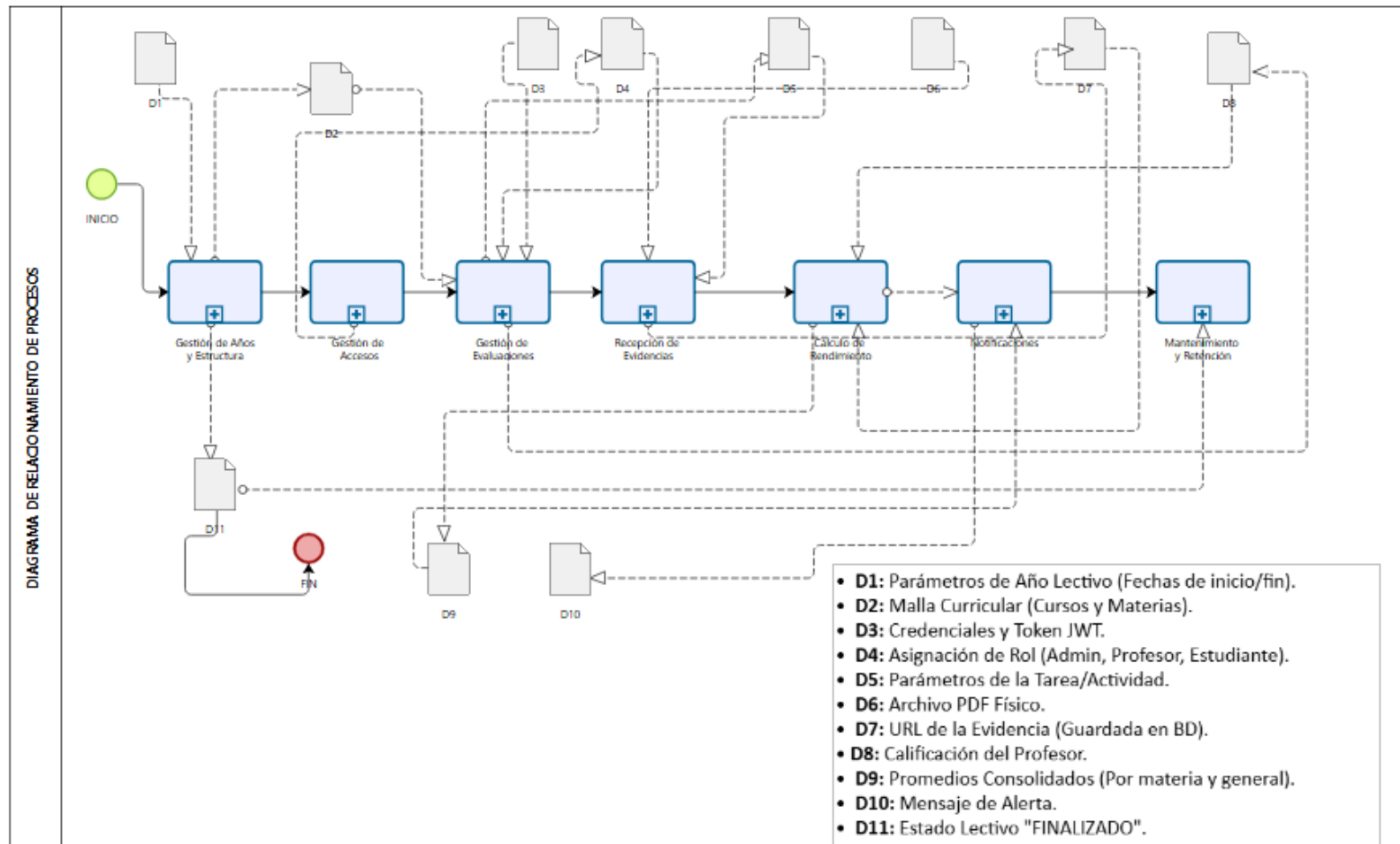
NORMA	AÑO	ARTÍCULOS	TEMAS
Estándares de Seguridad Web (OWASP)	2021	OWASP Top 10 (A07:2021)	Autenticación mediante tokens JWT y encriptación de contraseñas.
Normativa Interna de Gestión de Datos	2024	Políticas de Infraestructura TI	Política de retención de almacenamiento: Eliminación de evidencias físicas (PDFs) al finalizar el periodo lectivo.
Reglamento Educativo (Ecuador)	2023	Art. 18 al 22 (Sistema de Evaluación)	Estructuración académica basada en Años Lectivos, Cursos, Materias y división de evaluación en 3 Parciales.

3.2 DIRECTRICES DEL PROCESO

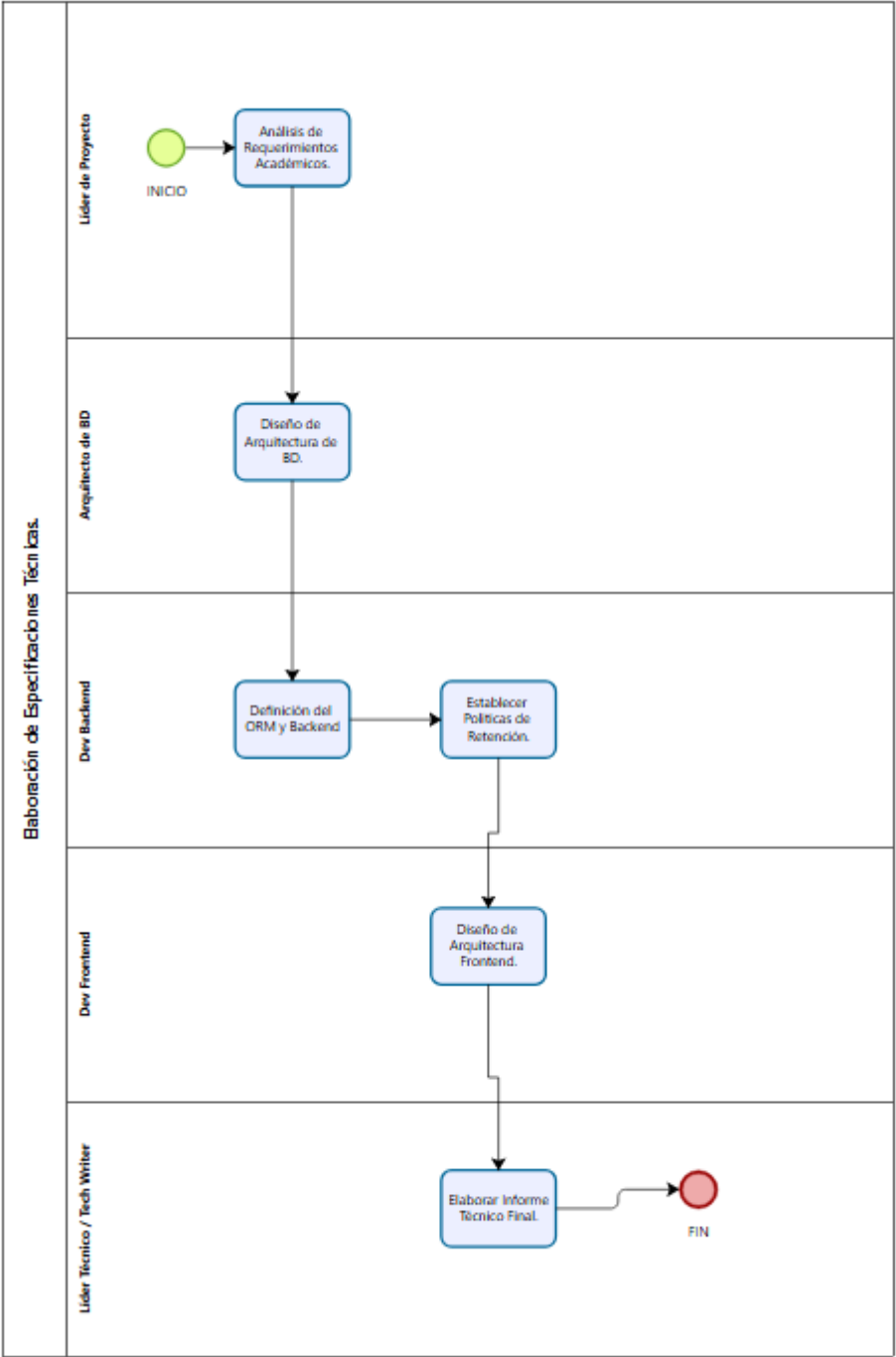
- a) Autenticación y Seguridad: El acceso al sistema se realizará validando el Correo Electrónico y contraseña. La sesión se manejará exclusivamente con JSON Web Tokens (JWT). No se contempla verificación de dos pasos (2FA).
- b) Control de Acceso Basado en Roles (RBAC): Se definen tres niveles de acceso inmutables en la base de datos: Administrador (Rol 1), Profesor (Rol 2) y Estudiante (Rol 3). El frontend deberá restringir las vistas según estos parámetros.
- c) Optimización de Almacenamiento: Bajo ninguna circunstancia se almacenarán archivos pesados (BYTEA/BLOB) en la base de datos. Los archivos de evidencias (PDF) se guardarán en un sistema de archivos externo o en la nube, almacenando únicamente su URL en la base de datos PostgreSQL.
- d) Política de Limpieza Automática: Se deberá programar un Cron Job o evento que ejecute el borrado físico de los archivos PDF una vez que el estado del año lectivo pase a "FINALIZADO", actualizando el registro en base de datos a "ELIMINADO" para liberar infraestructura.
- e) Experiencia de Usuario (UI/UX): Todos los formularios de operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en el frontend deberán desplegarse estrictamente como Modales (Pop-ups) utilizando Tailwind CSS, evitando la recarga de páginas independientes.

4 DESCRIPCIÓN DEL PROCESO

4.1 DIAGRAMA DE RELACIONAMIENTO DE LOS PROCESOS



4.2 DIAGRAMA DE FLUJO



4.3 DESCRIPCIÓN DE ACTIVIDADES

No.	ACTIVIDAD	DESCRIPCION DE LA ACTIVIDAD	RESPONSABLE	DOCUMENTO DE REFERENCIA/SISTEMA
1	Análisis de Requerimientos Académicos	Recepción y análisis de la lógica del sistema ecuatoriano (Cursos, Materias, 3 Parciales, Promedios).	Líder de Proyecto	Documento de Requisitos
2	Diseño de Arquitectura de Base de Datos	Definición del esquema relacional (15 tablas), normalización de la tabla EVIDENCIAS y configuración de PostgreSQL.	Arquitecto de Base de Datos	Esquema SQL / Diccionario de Datos
3	Definición del ORM y Backend	Integración de Prisma para la gestión de migraciones. Definición de módulos, controladores y servicios en Nest.js.	Desarrollador Backend Senior	Archivo schema.prisma
4	Establecimiento de Políticas de Retención	Diseño del algoritmo (<i>Cron Job</i>) para la depuración de archivos PDF al cierre de cada año lectivo.	Desarrollador Backend Senior	Directrices del Proceso
5	Diseño de Arquitectura Frontend	Definición del consumo de APIs, manejo de estado global para JWT y diseño de interfaces modales utilizando Next.js y Tailwind CSS.	Desarrollador Frontend Senior	Requerimientos de UI/UX
6	Elaboración de Informe Técnico Final	Consolidación de todas las especificaciones, endpoints (APIs) y modelo de datos en la documentación oficial para el equipo de desarrollo.	Líder Técnico / Technical Writer	Documentación Final (readme.md)

5 GLOSARIO DE TÉRMINOS Y ABREVIATURAS

TÉRMINO/ABREVIATURAS	DESCRIPCIÓN
CRUD	<i>Create, Read, Update, Delete</i> . Operaciones básicas de gestión de datos en el sistema.
JWT	<i>JSON Web Token</i> . Estándar utilizado para transmitir de forma segura la identidad y roles del usuario autenticado.
ORM	<i>Object-Relational Mapping</i> . Herramienta (en este caso, Prisma) que permite interactuar con la base de datos PostgreSQL utilizando código TypeScript en lugar de SQL puro.
Evidencia	Documento en formato PDF que un estudiante sube al sistema como resolución a una "Actividad" asignada por el profesor.
Modal / Pop-up	Ventana emergente en la interfaz de usuario superpuesta al contenido principal, utilizada para formularios.

6 ANEXOS

Anexo 1: Esquema de Base de Datos (schema.prisma optimizado).

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

// ===== ENUMS =====
enum EstadoUsuario {
  ACTIVO
  INACTIVO
  BLOQUEADO
}
```



```
enum EstadoLectivo {
    ACTIVO
    FINALIZADO
    PLANIFICADO
}
```

```
enum EstadoEvidencia {
    ACTIVO
    ELIMINADO
}
```

```
enum TipoActividad {
    TAREA
    EXAMEN
    PROYECTO
    LECCION
}
```

```
enum TipoNotificacion {
    NUEVA_ACTIVIDAD
    CALIFICACION
    RECORDATORIO
    SISTEMA
}
```

```
// ===== MODELOS =====
```

```
model AnioLectivo {
    idAnioLectivo Int      @id @default(autoincrement()) @map("id_aniolectivo")
    fechalnicio  DateTime  @map("fecha_inicio") @db.Date
    fechaFinal   DateTime  @map("fecha_final") @db.Date
    estadoLectivo EstadoLectivo @map("estado_lectivo")

    cursos      Curso[]
    promediosMat PromedioMateriaEstudiante[]

    @@map("anio_lectivo")
}
```

```
model Usuario {
    idUsuario      String  @id @map("id_usuario") @db.VarChar(10)
    nombreCompleto String  @map("nombre_completo") @db.VarChar(100)
    contrasenaUsuario String @map("contrasena_usuario") @db.VarChar(255)
    estadoUsuario  EstadoUsuario @map("estado_usuario")
}
```

```

email      String?    @unique @db.VarChar(255)
tokenRecuperacion String? @map("token_recuperacion") @db.VarChar(255)
expiracionToken DateTime? @map("expiracion_token")

roles      UsuarioRol[]
cursos     UsuarioCurso[]
docencias  ProfesorMateriaCurso[]
calificaciones Calificacion[]
evidencias Evidencia[]
notificaciones Notificacion[]
promediosMateria PromedioMateriaEstudiante[]
promediosGenerales PromedioGeneralEstudiante[]

@@map("usuario")
}

model Rol {
    idRol Int @id @default(autoincrement()) @map("id_rol")
    nombreRol String @map("nombre_rol") @db.VarChar(20)

    usuarios UsuarioRol[]

    @@map("rol")
}

model UsuarioRol {
    idUsuario String @map("id_usuario") @db.VarChar(10)
    idRol Int @map("id_rol")

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario], onDelete: Cascade)
    rol Rol @relation(fields: [idRol], references: [idRol])

    @@id([idUsuario, idRol])
    @@map("usuario_rol")
}

model Curso {
    idCurso Int @id @default(autoincrement()) @map("id_curso")
    idAnioLectivo Int @map("id_aniolectivo")
    nombreCurso String @map("nombre_curso") @db.VarChar(15)

    anioLectivo AnioLectivo @relation(fields: [idAnioLectivo], references:
[idAnioLectivo])

```

```

    estudiantes UsuarioCurso[]
    materias CursoMateria[]
    docentes ProfesorMateriaCurso[]
    parciales Parcial[]
    promediosMat PromedioMateriaEstudiante[]
    promediosGen PromedioGeneralEstudiante[]

    @@map("curso")
}

model UsuarioCurso {
    idUsuario String @map("id_usuario") @db.VarChar(10)
    idCurso Int @map("id_curso")

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario], onDelete: Cascade)
    curso Curso @relation(fields: [idCurso], references: [idCurso], onDelete: Cascade)

    @@id([idUsuario, idCurso])
    @@map("usuario_curso")
}

model Materia {
    idMateria Int @id @default(autoincrement()) @map("id_materia")
    nombreMateria String @map("nombre_materia") @db.VarChar(30)

    cursos CursoMateria[]
    parciales Parcial[]
    docentes ProfesorMateriaCurso[]
    promediosMat PromedioMateriaEstudiante[]

    @@map("materia")
}

model CursoMateria {
    idCurso Int @map("id_curso")
    idMateria Int @map("id_materia")

    curso Curso @relation(fields: [idCurso], references: [idCurso], onDelete: Cascade)
    materia Materia @relation(fields: [idMateria], references: [idMateria], onDelete: Cascade)

    @@id([idCurso, idMateria])
    @@map("curso_materia")
}

```

```
}
```

```
model ProfesorMateriaCurso {
  idUsuario String @map("id_usuario") @db.VarChar(10)
  idCurso Int @map("id_curso")
  idMateria Int @map("id_materia")

  usuario Usuario @relation(fields: [idUsuario], references: [idUsuario], onDelete: Cascade)
  curso Curso @relation(fields: [idCurso], references: [idCurso], onDelete: Cascade)
  materia Materia @relation(fields: [idMateria], references: [idMateria], onDelete: Cascade)

  @@id([idUsuario, idCurso, idMateria])
  @@map("profesor_materia_curso")
}
```

```
model Parcial {
  idParcial Int @id @default(autoincrement()) @map("id_parcial")
  idMateria Int @map("id_materia")
  idCurso Int @map("id_curso")
  numeroParcial Int @map("numero_parcial")

  materia Materia @relation(fields: [idMateria], references: [idMateria])
  curso Curso @relation(fields: [idCurso], references: [idCurso])
  actividades Actividad[]

  @@unique([idMateria, idCurso, numeroParcial], map:
"uq_parcial_materia_curso_numero")
  @@map("parcial")
}
```

```
model Actividad {
  idActividad Int @id @default(autoincrement()) @map("id_actividad")
  idParcial Int @map("id_parcial")
  tipoActividad TipoActividad @map("tipo_actividad")
  fechaInicioEntrega DateTime @map("fecha_inicio_entrega") @db.Date
  fechaFinEntrega DateTime @map("fecha_fin_entrega") @db.Date
  descripcion String? @db.VarChar(200)
  tituloActividad String? @map("titulo_actividad") @db.VarChar(20)
  valorMaximo Float @default(100.0) @map("valor_maximo")

  parcial Parcial @relation(fields: [idParcial], references: [idParcial])
  calificaciones Calificacion[]
}
```

```

evidencias Evidencia[]
notificaciones Notificacion[]

@@map("actividad")
}

model Calificacion {
    idCalificacion Int @id @default(autoincrement()) @map("id_calificacion")
    idUsuario String @map("id_usuario") @db.VarChar(10)
    idActividad Int @map("id_actividad")
    nota Float
    comentario String? @db.VarChar(200)

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario])
    actividad Actividad @relation(fields: [idActividad], references: [idActividad])

    @@unique([idUsuario, idActividad])
    @@map("calificacion")
}

model Evidencia {
    idEvidencia Int @id @default(autoincrement()) @map("id_evidencia")
    idUsuario String @map("id_usuario") @db.VarChar(10)
    idActividad Int @map("id_actividad")
    urlArchivo String? @map("url_archivo") @db.VarChar(255)
    nombreArchivo String @map("nombre_archivo") @db.VarChar(255)
    fechaSubida DateTime? @default(now()) @map("fecha_subida")
    tamaño BigInt
    tipoContenido String @map("tipo_contenido") @db.VarChar(255)
    estado EstadoEvidencia @default(ACTIVO)
    nombreActividad String @map("nombre_actividad") @db.VarChar(255)
    codigoActividad String @unique @map("codigo_actividad") @db.VarChar(255)
    tipoActividad String @map("tipo_actividad") @db.VarChar(255)

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario])
    actividad Actividad @relation(fields: [idActividad], references: [idActividad])

    @@unique([idUsuario, idActividad])
    @@map("evidencias")
}

model Notificacion {
    idNotificacion Int @id @default(autoincrement()) @map("id_notificacion")

```

```

idUsuario      String      @map("id_usuario") @db.VarChar(10)
idActividad    Int?        @map("id_actividad")
tipoNotificacion TipoNotificacion @map("tipo_notificacion")
mensajeNotificacion String    @map("mensaje_notificacion") @db.VarChar(255)
fechaNotificacion DateTime    @map("fecha_notificacion")
leida          Boolean      @default(false)

usuario Usuario @relation(fields: [idUsuario], references: [idUsuario])
actividad Actividad? @relation(fields: [idActividad], references: [idActividad])

@@map("notificaciones")
}

model PromedioMateriaEstudiante {
    idPromedioMateria      Int                @id @default(autoincrement())
    @map("id_promedio_materia")
    idUsuario              String @map("id_usuario") @db.VarChar(10)
    idAnioLectivo          Int    @map("id_aniolectivo")
    idCurso                Int    @map("id_curso")
    idMateria              Int    @map("id_materia")
    promedioParcial1       Float? @map("promedio_parcial1")
    promedioParcial2       Float? @map("promedio_parcial2")
    promedioParcial3       Float? @map("promedio_parcial3")
    promedioFinalMateria   Float? @map("promedio_final_materia")
    fechaActualizacion     DateTime? @map("fecha_actualizacion")

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario])
    anioLectivo AnioLectivo @relation(fields: [idAnioLectivo], references: [idAnioLectivo])
    curso Curso @relation(fields: [idCurso], references: [idCurso])
    materia Materia @relation(fields: [idMateria], references: [idMateria])

    @@unique([idUsuario, idAnioLectivo, idCurso, idMateria])
    @@map("promedio_materia_estudiante")
}

model PromedioGeneralEstudiante {
    idPromedioGeneral Int @id @default(autoincrement()) @map("id_promedio_general")
    idUsuario          String @map("id_usuario") @db.VarChar(10)
    idCurso            Int @map("id_curso")
    promedioGeneral    Float @map("promedio_general")
    comportamiento     String @db.VarChar(1)

    usuario Usuario @relation(fields: [idUsuario], references: [idUsuario])

```

```
curso Curso @relation(fields: [idCurso], references: [idCurso])
```

```
@@unique([idUsuario, idCurso])
```

```
@@map("promediogeneralestudiante")
```

```
}
```

Anexo 2:  Colección de Endpoints API REST (Rutas, métodos y Payloads JSON).

Auth

POST `/api/auth/login`

Obtiene el JWT. **No requiere autenticación.**

Admin:

```
{
  "email": "admin@notas.edu.ec",
  "password": "Admin#2026"
}
```

Profesor:

```
{
  "email": "profesor@notas.edu.ec",
  "password": "Profesor#2026"
}
```

Estudiante:

```
{
  "email": "estudiante@notas.edu.ec",
  "password": "Estudiante#2026"
}
```

Respuesta exitosa:

```
{
  "access_token": "eyJhbGciOi...",
  "idUsuario": "0000000001",
  "nombreCompleto": "Administrador del Sistema",
  "roles": [1]
}
```

POST `/api/auth/forgot-password`

Solicita recuperación de contraseña por email.

```
{
  "email": "profesor@notas.edu.ec"
}
```

POST `/api/auth/reset-password`

Restablece la contraseña con el token recibido por email.

```
{
  "token": "f3a9b1c2-...-xyz",
  "newPassword": "NuevaClave#2026"
}
```

Usuarios (solo ADMIN)

POST `/api/usuarios`

Crea un nuevo usuario.

```
{
  "idUsuario": "0102030400",
  "nombreCompleto": "María López Pérez",
  "contrasenaUsuario": "Clave#2026",
  "email": "maria.lopez@notas.edu.ec",
  "estadoUsuario": "ACTIVO",
  "roles": [2]
}
```

estadoUsuario: ACTIVO | INACTIVO | BLOQUEADO